

Relatório Técnico: Análise de Erros no Sistema OpenCargo

Data: 08 de julho de 2026

Autor: Manus AI

1. Introdução

Este relatório técnico apresenta uma análise comparativa entre o comportamento observado na aplicação web OpenCargo (versão hospedada) e seu código-fonte disponível no GitHub. O objetivo é identificar as causas técnicas dos erros funcionais e de exibição de dados previamente documentados.

2. Sumário dos Erros Observados

Durante a fase de testes exploratórios na aplicação hospedada, foram identificados os seguintes problemas:

- **Erros de Carregamento de Página:** As páginas de Dashboard, Matching, Chat, Settings e Profile falharam ao carregar, exibindo a mensagem "Erro ao carregar página: Não foi possível carregar '[nome da página]'. Tente novamente."
- **Inconsistência na Exibição de Dados - Veículos:** Na seção Veículos, a coluna "Status" para todos os itens listados apresentava o valor "undefined".
- **Inconsistência na Exibição de Dados - Cargas:** Na seção Cargas, a coluna "Empresa" estava vazia para todas as cargas.

3. Análise do Código-Fonte do Frontend

A análise do diretório frontend (/home/ubuntu/opencargo_repo/frontend) revelou a seguinte estrutura e lógica:

3.1. Roteamento (router.js)

O arquivo router.js 1 é responsável pelo roteamento client-side, carregando as páginas dinamicamente com base na hash da URL. A função _loadPage (linhas 119-184) mapeia nomes de página (ex: "dashboard", "matching") para módulos JavaScript correspondentes (ex: DashboardPage, MatchingPage). Em caso de erro no carregamento da página, a função _errorState (linhas 218-231) é invocada, exibindo a mensagem de erro genérica observada.

3.2. Serviço de API (api.js)

O arquivo `api.js` 2 centraliza as chamadas à API. A lógica de `Api.get(endpoint)` (linhas 19-44) verifica a variável `CONFIG.API_BASE_URL`. Se esta estiver vazia, o frontend utiliza dados mockados (`_loadMock` nas linhas 96-110) de arquivos JSON locais (ex: `data/loads.json`). Se `API_BASE_URL` estiver preenchida, ele tenta fazer requisições reais ao backend.

3.3. Configuração de Ambiente (`env.js`)

O arquivo `env.js` 3 define a `API_BASE_URL`. Na versão do repositório, a URL base da API está configurada como `https://opencargo-production.up.railway.app/api` (linha 25). Isso indica que a aplicação está configurada para se comunicar com um backend real, e não com dados mockados.

3.4. Lógica das Páginas (Ex: `dashboard.js`, `matching.js`, `chat.js`, `profile.js`, `settings.js`)

As páginas que apresentaram erros de carregamento (`dashboard`, `matching`, `chat`, `profile`, `settings`) compartilham um padrão: elas tentam buscar dados usando `Api.get("nome_da_pagina")` ou `Api.get("recurso")`.

- **`dashboard.js` 4:** A função `render` (linhas 11-60) tenta carregar dados de várias APIs, como `Api.get("companies")`, `Api.get("drivers")`, `Api.get("loads")`, `Api.get("routes")`, `Api.get("matches")`.
- **`matching.js` 5:** A função `render` (linhas 31-43) faz chamadas para `Api.get("matches")` e `Api.get("matching/filters")`, e a função `_search` (linhas 60-75) chama `Api.get("matching/search?...")`.
- **`chat.js` 6:** A função `render` (linhas 11-18) tenta buscar `Api.get("messages")` e `Api.get("matches")`.
- **`profile.js` 7:** A página de perfil busca dados usando `/auth/me` e `PATCH /users/:id`.
- **`settings.js` :** Embora não tenhamos lido o conteúdo específico, é provável que siga um padrão similar de busca de dados via `Api.get`.

4. Análise do Código-Fonte do Backend

A análise do diretório `backend` (`/home/ubuntu/opencargo_repo/backend`) e, especificamente, do arquivo `app.js` 8 (linhas 97-110), que registra as rotas da API, revelou o seguinte:

4.1. Rotas da API

O backend utiliza o framework Fastify e registra rotas com prefixos específicos:

- `/api/auth`

- `/api/users`
- `/api/companies`
- `/api/drivers`
- `/api/vehicles`
- `/api/routes`
- `/api/loads`
- `/api/matching`
- `/api/maps`
- `/api/notifications`
- `/api/documents`
- `/api/reviews`
- `/api/freights`
- `/api/chat`

É crucial notar que não existem rotas registradas diretamente como `/api/dashboard`, `/api/matches` (na raiz), `/api/messages` (na raiz), `/api/settings` ou `/api/profile`.

4.2. Schema do Banco de Dados (`common/database.js`)

O arquivo `common/database.js` [9](#) define o schema SQLite do projeto.

- **Tabela `vehicles`** : A definição da tabela `vehicles` (linhas 125-135) inclui colunas como `plate`, `model`, `year`, `capacity_kg`, `capacity_m3` e `type`, mas **não possui uma coluna `status`**.
- **Tabela `loads`** : A tabela `loads` (linhas 153-170) armazena `company_id`, mas **não `company_name`**.

4.3. Rotas de Veículos (`vehicles/routes.js`)

O endpoint `GET /` em `vehicles/routes.js` [10](#) (linhas 38-47) retorna todos os veículos, mas como o schema do banco de dados não inclui `status`, essa informação não é fornecida pelo backend.

4.4. Rotas de Cargas (`loads/routes.js`)

O endpoint `GET /` em `loads/routes.js` [11](#) (linhas 44-53) realiza uma consulta `SELECT * FROM loads` sem realizar um `JOIN` com a tabela `companies`. Consequentemente, o nome da empresa não é retornado junto com os dados da carga.

5. Comparação e Causas Raiz dos Erros

5.1. Erros de Carregamento de Página

A causa raiz dos erros de carregamento de página (`Dashboard` , `Matching` , `Chat` , `Settings` , `Profile`) é uma **incompatibilidade entre os endpoints de API esperados pelo frontend e as rotas realmente expostas pelo backend**.

- O frontend, através de `api.js` , tenta acessar endpoints como `Api.get("dashboard")` , `Api.get("matches")` , `Api.get("messages")` , `Api.get("settings")` , `Api.get("profile")` .
- O backend, conforme `app.js` , não possui rotas diretas para `dashboard` , `matches` , `messages` , `settings` ou `profile` na raiz `/api/` . Em vez disso, ele tem rotas como `/api/matching` (para recursos de matching), `/api/chat` (para recursos de chat), `/api/users` (que pode conter dados de perfil) e `/api/auth` (para autenticação).

Quando o frontend tenta chamar `Api.get("dashboard")` , por exemplo, ele espera que o backend responda a `/api/dashboard` . No entanto, o backend não tem essa rota, resultando em um erro HTTP (provavelmente 404 Not Found ou similar), que é capturado pelo bloco `catch` em `router.js` (linhas 85-94), levando à exibição da mensagem "Erro ao carregar página". O mesmo se aplica a `matching` , `chat` , `settings` e `profile` (que provavelmente deveriam buscar dados via `/api/users` ou `/api/auth/me`).

5.2. Inconsistência na Exibição de Dados - Veículos

O problema da coluna "Status" como "undefined" na seção `Veículos` é causado pela **ausência da coluna `status` na tabela `vehicles` do banco de dados** 9. O frontend espera um campo `status` para exibir, mas o backend não o fornece, pois ele não existe no schema da tabela nem é adicionado na consulta de retorno 10.

5.3. Inconsistência na Exibição de Dados - Cargas

A coluna "Empresa" vazia na seção `Cargas` ocorre porque a **consulta do backend para cargas não inclui o nome da empresa** 11. A tabela `loads` armazena apenas o `company_id` 9. Para exibir o nome da empresa, o backend precisaria realizar um `JOIN` com a tabela `companies` ao buscar os dados das cargas, ou o frontend precisaria fazer uma requisição adicional para cada `company_id` para obter o nome, o que seria ineficiente.

6. Recomendações

Para resolver os problemas identificados, as seguintes ações são recomendadas:

6.1. Correção dos Endpoints da API (Frontend e Backend)

- **Frontend:** Ajustar as chamadas `Api.get()` em `dashboard.js`, `matching.js`, `chat.js`, `profile.js` e `settings.js` para corresponderem às rotas reais do backend. Por exemplo, para o dashboard, pode ser necessário chamar múltiplos endpoints (ex: `Api.get("companies")`, `Api.get("loads")`) e consolidar os dados no frontend. Para `matching` e `chat`, as chamadas já estão usando `matching/search` e `chat/messages` respectivamente, mas `Api.get("matches")` e `Api.get("messages")` na raiz ainda causam problemas. O `router.js` também precisa ser revisado para garantir que os nomes das páginas mapeiem corretamente para os recursos da API ou que a lógica de carregamento de dados seja mais robusta.
- **Backend:** Garantir que todas as rotas necessárias para o frontend estejam devidamente expostas e que os dados retornados correspondam ao que o frontend espera. Se o frontend espera um endpoint `/api/dashboard` que consolide dados, essa rota precisa ser criada no backend.

6.2. Adição do Campo `status` na Tabela `vehicles`

- **Backend (Banco de Dados):** Adicionar uma coluna `status` à tabela `vehicles` no schema do banco de dados (`common/database.js`) e garantir que ela seja preenchida e atualizada corretamente. Isso pode envolver a criação de um novo campo no `createVehicleSchema` e a inclusão do `status` nas operações de `INSERT` e `UPDATE` em `vehicles/routes.js`.
- **Frontend:** Atualizar a lógica de exibição na página `vehicles.js` para usar o novo campo `status` corretamente.

6.3. Inclusão do Nome da Empresa na Consulta de Cargas

- **Backend:** Modificar a consulta `GET /` em `loads/routes.js` para incluir um `JOIN` com a tabela `companies`, retornando o `company_name` junto com os dados da carga. Isso permitirá que o frontend exiba o nome da empresa diretamente.

Referências

Footnotes

1. [router.js](#) ↩
2. [api.js](#) ↩
3. [env.js](#) ↩
4. [dashboard.js](#) ↩
5. [matching.js](#) ↩

6. [chat.js](#) ↩
7. [profile.js](#) ↩
8. [app.js](#) ↩
9. [database.js](#) ↩ ↩2 ↩3
10. [vehicles/routes.js](#) ↩ ↩2
11. [loads/routes.js](#) ↩ ↩2